

EE 465 Final Project

Date: 12/11/2024

By: Tanvi Mehetre & Olivia Price

Contributions

Tanvi: coder and optimizer

Olivia: coder and optimizer

Part 2 Introduction:

For this project, we were instructed to aid the United States Department of Agriculture. The department will give us samples of temperature every so often, and we are instructed to find the moving average and moving standard deviation. To get started, we will create a code in Verilog where the inputs include mode, reset, temperature, and clock, and the outputs include sample, done, and standard deviation/average. Depending on the mode for the specific input, we will output the moving average if the mode is zero and the standard deviation if the mode is one. The done output is to tell the code when its production is ready and the sample output tells the code when it is prepared for a new sample.

Contributions of Team Members:

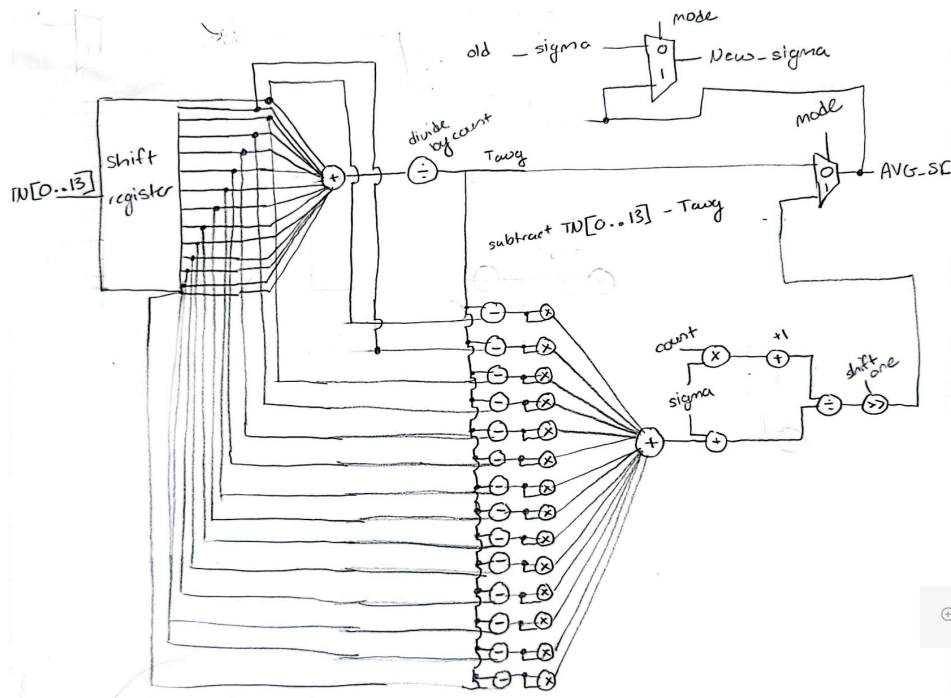
Both team members worked on coding the verilog for the moving standard deviation and moving average. Then, for optimization, Tanvi did the RTL synthesizing, and Olivia did the Innovus optimization. Then, we both worked on the report together.

Part 3: Base Code and Optimized Code

Introduction:

For this code, we decided to code the project simply and to get it working. We wanted to provide a base that we could optimize. For the code below, we created a shift register that shifted in a new value every clock cycle and outputted the standard deviation or moving average. To account for the rounding errors, we divided the count by two and then added it to the sum. This ensured that the output matched our expected output.

Schematic:



Code:

```

`timescale 1ns/1ns

module final_project_final(
    input CLK,
    input RESET,
    input [11:0] TN,
    input MODE,
    output SAMPLE,
    output DONE,
    output [11:0] AVG_SD
);

reg [11:0] sigma_hat;
reg [11:0] Tvals [0:13];

reg mode;
integer i;

reg [8:0] count;
reg [11:0] Tavg;
reg [31:0] sum1;
reg [31:0] sum2;

reg [13:0] sd;
reg [11:0] avg_sd;
reg done;
reg sample;

reg [31:0] sumTemp [13:0];

initial sigma_hat = 12'b010000000000;
initial done = 1'b0;
initial sum1 = 32'b0;
initial sum2 = 32'b0;

always@(CLK)begin
    sample <= (TN > 0 && TN != Tvals[0]);
end

assign AVG_SD = avg_sd;
assign DONE = done;
assign SAMPLE = sample;

```

```

always@(posedge CLK) begin
  if(RESET) begin
    Tvals[13] <= 14'b0;
    Tvals[12] <= 14'b0;
    Tvals[11] <= 14'b0;
    Tvals[10] <= 14'b0;
    Tvals[9] <= 14'b0;
    Tvals[8] <= 14'b0;
    Tvals[7] <= 14'b0;
    Tvals[6] <= 14'b0;
    Tvals[5] <= 14'b0;
    Tvals[4] <= 14'b0;
    Tvals[3] <= 14'b0;
    Tvals[2] <= 14'b0;
    Tvals[1] <= 14'b0;
    Tvals[0] <= 14'b0;
    done <= 1'b0;
    sum1 <= 32'b0;
    sum2 <= 32'b0;
    sigma_hat <= 12'b010000000000;

  end
  else begin

    done <= 1'b0;
    mode <= MODE;

    if (SAMPLE) begin
      for ( i = 13; i >= 1; i = i - 1) begin
        Tvals[i] <= Tvals[i - 1];
      end
      Tvals[0] <= TN;
    end

    sum1 = 32'b0;
    sum2 = 32'b0;
    count = 9'b0;

    for ( i = 13; i >= 0 ; i = i - 1) begin
      if (Tvals[i] != 0)
        count = count + 1;
    end
  end
end

```

```

end

if(count > 0)begin
    sum1 = Tvals[0] + Tvals[1] + Tvals[2] + Tvals[3] + Tvals[4] + Tvals[5] +
Tvals[6] +
        Tvals[7] + Tvals[8] + Tvals[9] + Tvals[10] + Tvals[11] + Tvals[12] +
Tvals[13];

    Tavg = (sum1 + (count / 2)) / (count);

    for (i = 13; i >= 0; i = i - 1) begin
        if (Tvals[i] != 0)
            sumTemp[i] = (Tvals[i] - Tavg) * (Tvals[i] - Tavg);
        else
            sumTemp[i] = 0;
        end

    for ( i = 13; i >= 0 ; i = i - 1) begin

        sum2 = sum2 + sumTemp[i];

    end

    sd = (sigma_hat + sum2 / (count * sigma_hat) + 1) / 2;

    $monitor("sum1 = %d, sum2 = %d, count = %d, sigma_hat = %d", sum1,
sum2, count, sigma_hat);

    avg_sd <= !mode ? Tavg : sd;
    done <= 1'b1;

    if (mode && SAMPLE) begin
        sigma_hat = sd;
    end

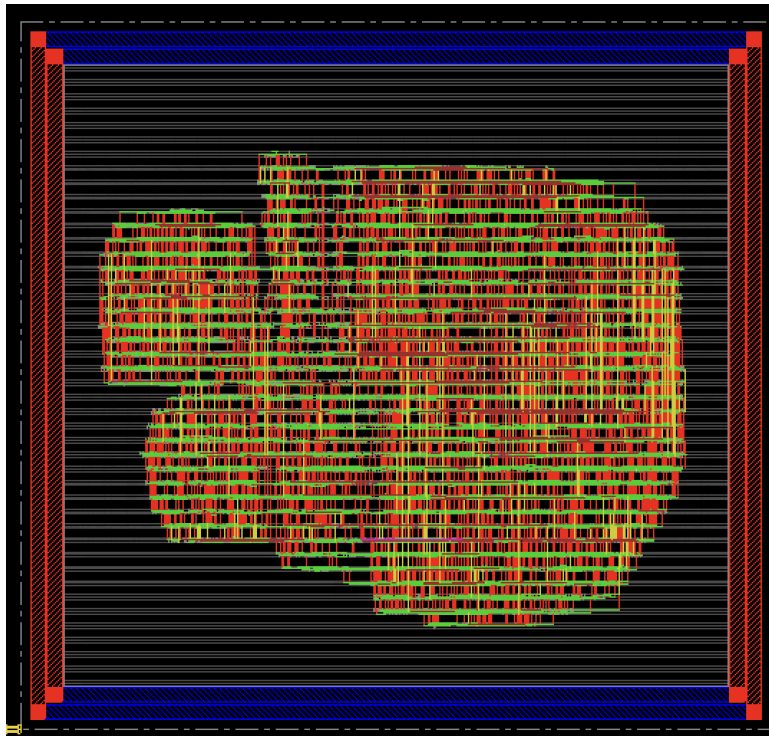
end
end
end
endmodule

```

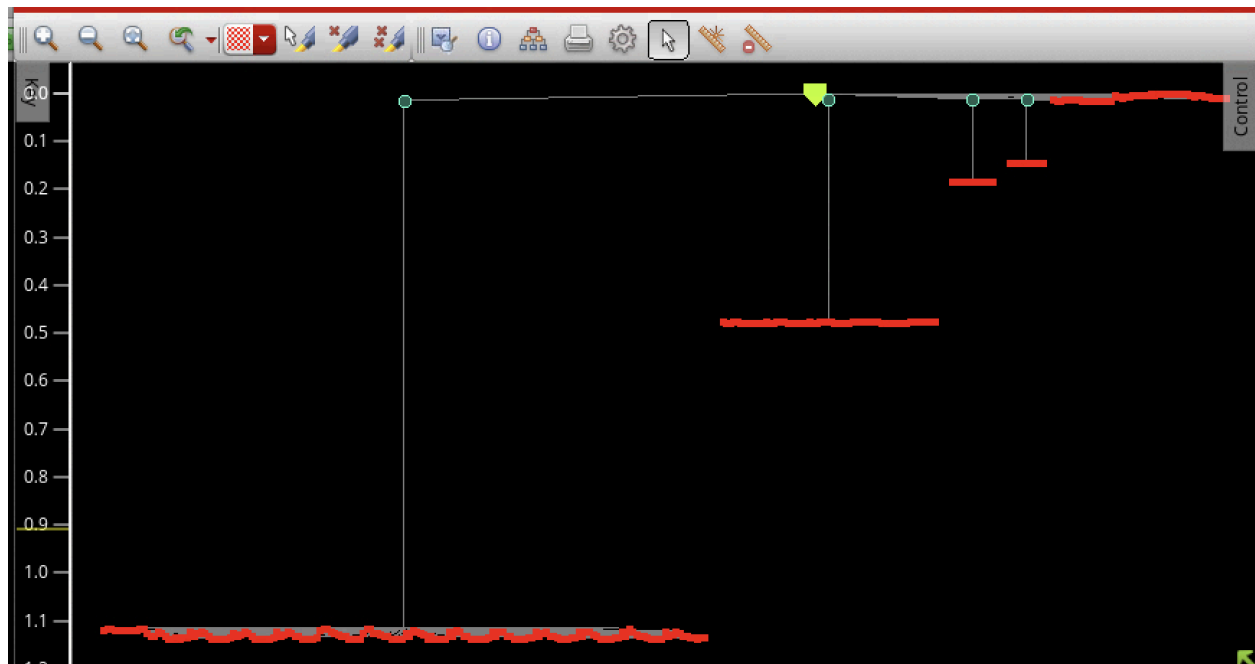
Results:



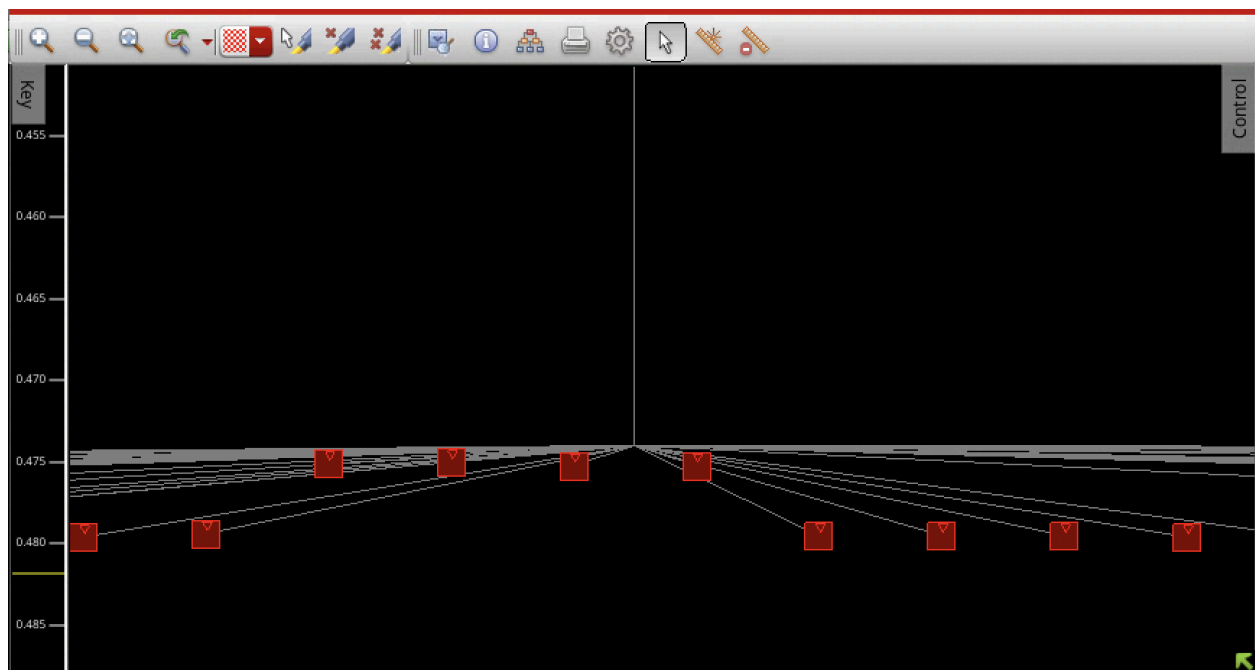
Layout:



Clock Tree:



Zoomed in on the middle of the tree



Timing report for innovus

```

Path 1: MET Setup Check with Pin Tavg_reg[0]/CP
Endpoint:  Tavg_reg[0]/D (^) checked with leading edge of 'CLK'
Beginpoint: count_reg[1]/Q (^) triggered by leading edge of 'CLK'
Path Groups: {CLK}
Analysis View: typical_view
Other End Arrival Time          0.000
- Setup                        0.045
+ Phase Shift                  15.400
= Required Time                15.355
- Arrival Time                 15.190
= Slack Time                   0.165
    Clock Rise Edge            0.000
    + Clock Network Latency (Ideal) 0.000
    = Beginpoint Arrival Time  0.000

```

Area report for innovus

```

innovus 3> report_area

```

Hinst Name	Module Name	Inst Count	Total Area
final_project_final		9620	29017.080
RC_CG_HIER_INST0	RC_CG_MOD	1	6.480
RC_CG_HIER_INST1	RC_CG_MOD_1	1	6.480
RC_CG_HIER_INST2	RC_CG_MOD_2	1	6.480
RC_CG_HIER_INST3	RC_CG_MOD_3	1	6.480

Power report for innovus

```

-----
*      Power Distribution Summary:
*      Highest Average Power:          FE_RC_1454_0 (CKND16):          0.005437
*      Highest Leakage Power: FE_0FC211_div_105_28_n_933 (BUFFD16):    0.0002565
*      Total Cap: 7.52107e-11 F
*      Total instances in design: 9620
*      Total instances in design with no power: 0
*      Total instances in design with no activity: 0
*
*      Total Fillers and Decap: 0
-----

```

```

Total Power
-----
Total Internal Power: 1.47403793 38.4896%
Total Switching Power: 2.03996737 53.2669%
Total Leakage Power: 0.31570080 8.2435%
Total Power: 3.82970610
-----

```

Timing Report


```

(clock CLK)                                capture                                15400 R
-----
Cost Group   : 'CLK' (path_group 'CLK')
Timing slack :      0ps
Start-point  : sigma_hat_reg[0]/CP
End-point    : sd_reg[11]/SE

```

Area Report

```

legacy_genus:/> report area
=====
Generated by:      Genus(TM) Synthesis Solution 22.16-s078_1
Generated on:      Dec 11 2024 11:38:12 am
Module:            final_project_final
Technology library: tcbn65gpluswc_121
Operating conditions: WCCOM (balanced_tree)
Wireload mode:     segmented
Area mode:         timing library
=====

Instance      Module  Cell Count  Cell Area  Net Area  Total Area  Wireload
-----
final_project_final  7261  18268.560    0.000    18268.560 ZeroWireload (S)
(S) = wireload was automatically selected

```

Power Report

```

legacy_genus:/> report power
Info      : Joules engine is used. [RPT-16]
           : Joules engine is being used for the command report_power.
Instance:  /final_project_final
Power Unit: W
PDB Frames: /stim#0/frame#0
-----
Category      Leakage      Internal      Switching      Total      Row%
-----
memory        0.000000e+00  0.000000e+00  0.000000e+00  0.000000e+00  0.00%
register      1.76925e-05   8.77974e-05   1.05824e-05   1.16072e-04   11.40%
latch         0.000000e+00  0.000000e+00  0.000000e+00  0.000000e+00  0.00%
logic         1.50317e-04   3.90582e-04   3.53834e-04   8.94733e-04   87.89%
bbox          0.000000e+00  0.000000e+00  0.000000e+00  0.000000e+00  0.00%
clock         1.65098e-07   1.72849e-06   5.35918e-06   7.25276e-06   0.71%
pad           0.000000e+00  0.000000e+00  0.000000e+00  0.000000e+00  0.00%
pm            0.000000e+00  0.000000e+00  0.000000e+00  0.000000e+00  0.00%
-----
Subtotal      1.68175e-04   4.80108e-04   3.69775e-04   1.01806e-03  100.00%
Percentage    16.52%       47.16%       36.32%       100.00%  100.00%

```

Clock Period (ns)	15.4
Slack (ns)	0 ps

Average Power Consumption (mW)	1.018
Area (micrometer ²)	18268.560
average energy consumption per temperature reading(mW)	0.0339
latency (i.e., the # of clock cycles it takes to generate AVG/SD after SAMPLE is asserted)	1.5 cycles
Throughput (i.e., the maximum # of temperature readings the circuit can handle in 1 second)	23.1

Optimized Code/Finite State Machine:

Introduction: To optimize our code, we created a finite-state machine to decrease our area and power consumption. Finite-state machines help minimize switching power, low-power idle states, clock gating, and reduced combinational logic complexity. This also decreased area because it reuses logic and has efficient logic representation.

Code:

```
`timescale 1ns/1ns

module final_project_fsmm(
    input CLK,
    input RESET,
    input [11:0] TN,
    input MODE,
    output SAMPLE,
    output DONE,
    output [11:0] AVG_SD
);

// State encoding
localparam IDLE = 2'b00;
localparam SAMPLE_STATE = 2'b01;
localparam CALCULATE = 2'b10;
localparam DONE_STATE = 2'b11;

// Registers and wires
reg [11:0] sigma_hat;
reg [11:0] Tvals [0:13];
```

```

reg [1:0] state, next_state;

reg [8:0] count;
reg [11:0] Tavg;
reg [31:0] sum1;
reg [31:0] sum2;

reg mode;

reg [11:0] sd;
reg [11:0] avg_sd;
reg done;
reg sample;

reg [31:0] sumTemp [13:0];
integer i;

// Outputs
assign AVG_SD = avg_sd;
assign DONE = done;
assign SAMPLE = sample;

// Initial values
initial begin
    sigma_hat = 12'b010000000000; // 32°F
    done = 1'b0;
    state = IDLE;
    for (i = 0; i < 14; i = i + 1)
        Tvals[i] <= 12'b0;
    sum1 <= 32'b0;
    sum2 <= 32'b0;
    avg_sd <= 12'b0;
    count <= 9'b0;
end

// State transition logic
always @(posedge CLK or posedge RESET) begin
    if (RESET)
        state <= IDLE;
    else
        state <= next_state;
end

// Next state and output logic
always @(*) begin
    // Default assignments
    next_state = state;
    sample = 1'b0;
    done = 1'b0;
    mode <= MODE;

```

```

case (state)
IDLE: begin
if (TN > 0) begin
    sample = 1'b1;
    next_state = SAMPLE_STATE;
    $display("Transitioning to SAMPLE_STATE");
end
end

SAMPLE_STATE: begin
next_state = CALCULATE;
$display("Transitioning to CALCULATE");
end

CALCULATE: begin
next_state = DONE_STATE;
$display("Transitioning to DONE_STATE");
end

DONE_STATE: begin
done = 1'b1; // Ensure DONE is set here
next_state = SAMPLE_STATE; // Restart from SAMPLE_STATE
$display("DONE_STATE: DONE = %b", done);
end

default: next_state = IDLE;
endcase
end

// Computation logic
always @(posedge CLK or posedge RESET) begin
if (RESET) begin
    // Initialize all values
    for (i = 0; i < 14; i = i + 1)
        Tvals[i] <= 12'b0;
    sigma_hat <= 12'b010000000000; // 32°F
    sum1 <= 32'b0;
    sum2 <= 32'b0;
    avg_sd <= 12'b0;
    count <= 9'b0;
    done <= 1'b0;
end else begin
case (state)
SAMPLE_STATE: begin
    // Shift values and store the new temperature
    for (i = 13; i > 0; i = i - 1)
        Tvals[i] <= Tvals[i - 1];
    Tvals[0] <= TN;
    $display("SAMPLE_STATE: TN = %d", TN);

```

```

end

CALCULATE: begin

    sum1 = 32'b0;
sum2 = 32'b0;
    count = 9'b0;

    // Count non-zero values and sum temperatures
    for (i = 13; i >= 0; i = i - 1) begin
        if (Tvals[i] != 0) begin
            //sum1 = sum1 + Tvals[i];
            count = count + 1;
        end
    end

    if (count > 0) begin

        sum1 = Tvals[0] + Tvals[1] + Tvals[2] + Tvals[3] + Tvals[4] + Tvals[5] + Tvals[6]
+
        Tvals[7] + Tvals[8] + Tvals[9] + Tvals[10] + Tvals[11] + Tvals[12] +
Tvals[13];
        Tavg = (sum1 + (count / 2)) / (count);

        // Calculate standard deviation if MODE is 1

        for (i = 13; i >= 0; i = i - 1) begin
            if (Tvals[i] != 0)
                sumTemp[i] = (Tvals[i] - Tavg) * (Tvals[i] - Tavg);
            else
                sumTemp[i] = 0;
        end

        for (i = 13; i >= 0; i = i - 1) begin
            sum2 = sum2 + sumTemp[i];
        end

        sd = (sigma_hat + sum2 / (count * sigma_hat) + 1) / 2;

        avg_sd <= !mode ? Tavg : sd;

        $monitor("sum1 = %d, sum2 = %d, count = %d, sigma_hat = %d", sum1,
sum2, count, sigma_hat);

        done <= 1'b1;

        if (mode) begin

```

```

        sigma_hat = sd;
        end
        end

    end

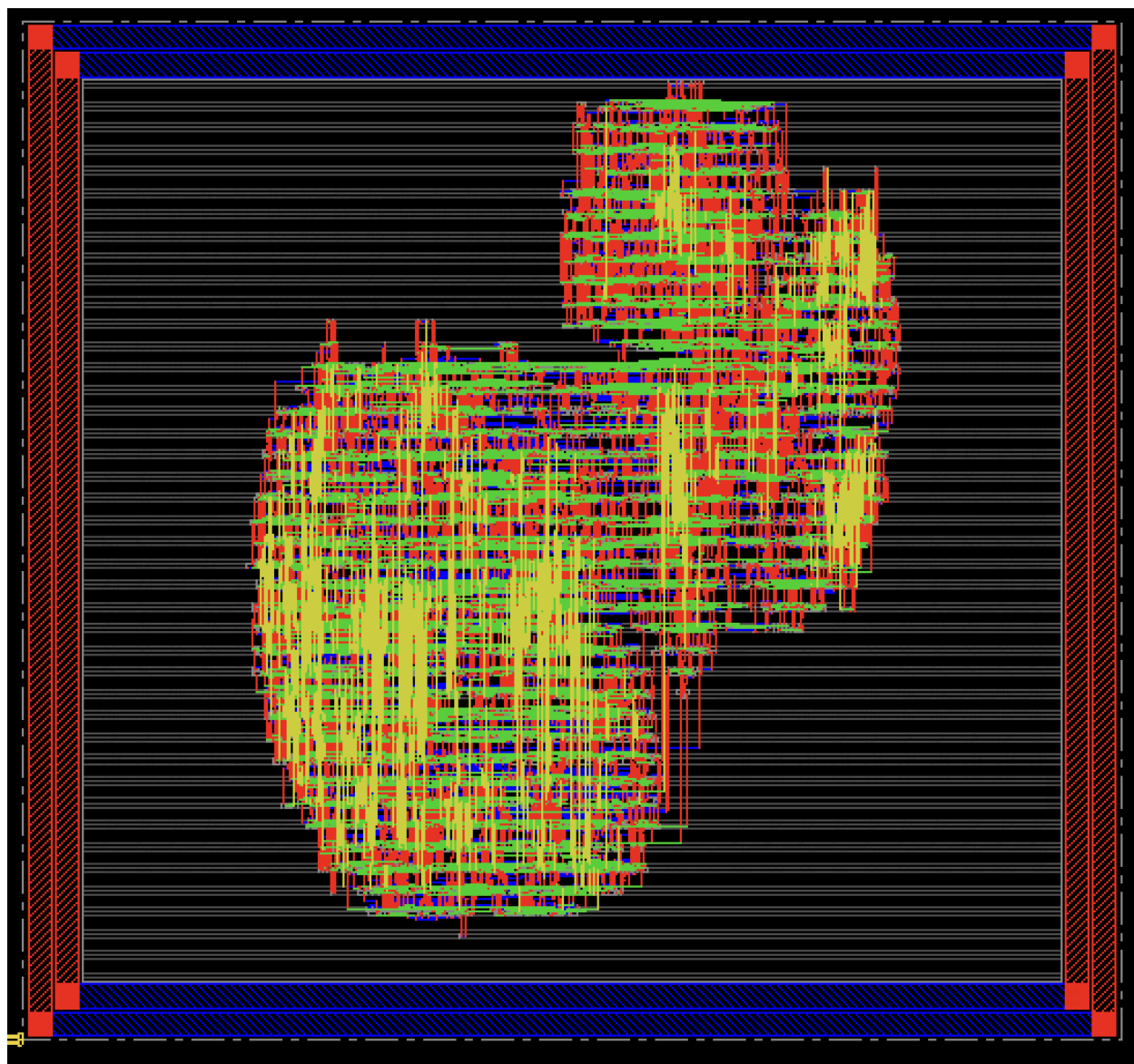
    DONE_STATE: begin
        // Ensure DONE is set and output is generated
        done = 1'b1;
        $display("DONE_STATE: DONE = %b, AVG_SD = %d", done, avg_sd);
    end
    endcase
end
endmodule

```

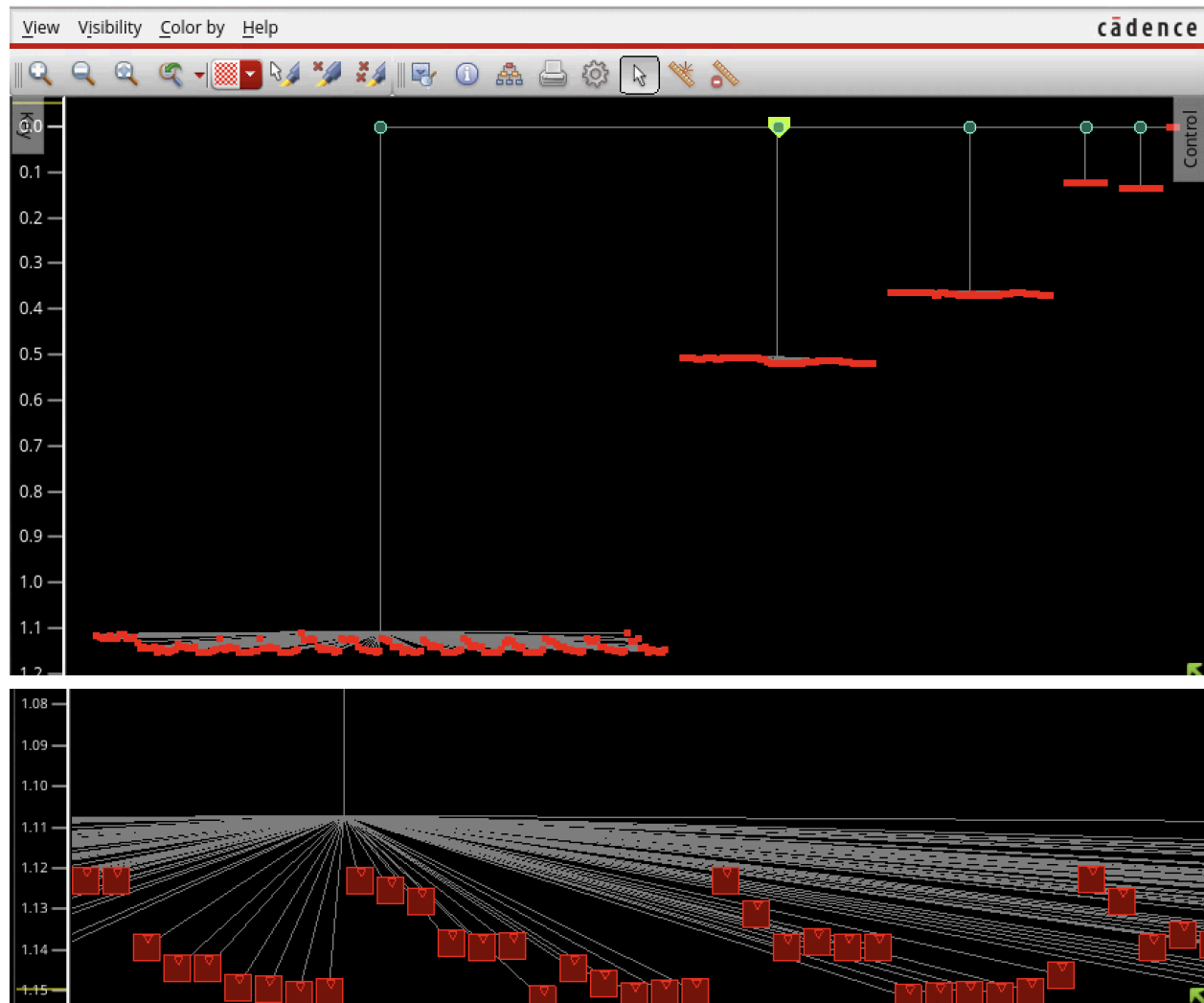
Results:



Layout:



Clock Tree



Timing report for Innovus


```

Path Groups: {CLK}
Analysis View: typical_view
Other End Arrival Time          0.000
- Setup                        0.190
+ Phase Shift                  20.000
= Required Time                19.810
- Arrival Time                 19.611
= Slack Time                   0.199
    Clock Rise Edge            0.000
    + Clock Network Latency (Ideal) 0.000
    = Beginpoint Arrival Time   0.000

```

Area report for Innovus

```

innovus 3> report_area

```

Hinst Name	Module Name	Inst Count	Total Area
final_project_fsmm		6820	18969.120
RC_CG_HIER_INST0	RC_CG_MOD	1	6.480
RC_CG_HIER_INST1	RC_CG_MOD_1	1	6.480
RC_CG_HIER_INST2	RC_CG_MOD_2	1	6.480
RC_CG_HIER_INST3	RC_CG_MOD_3	1	6.480
RC_CG_HIER_INST4	RC_CG_MOD_4	1	6.480

```

innovus 4>

```

Power report for innovus

```

-----
*      Power Distribution Summary:
*      Highest Average Power:      div_159_49_g90977 (CKX0R2D1):      0.002591
*      Highest Leakage Power:      g91792 (CMPE42D1):      0.000241
*      Total Cap:      4.16047e-11 F
*      Total instances in design: 6820
*      Total instances in design with no power:      0
*      Total instances in design with no activity:      0
*
*      Total Fillers and Decap:      0
-----
** INFO: (VOLTUS_POWR-3465): There are 0 decaps and 0 fillers in the design

Ended Static Power Report Generation: (cpu=0:00:00, real=0:00:00,
mem(process/total/peak)=2614.29MB/4698.06MB/2614.29MB)

```

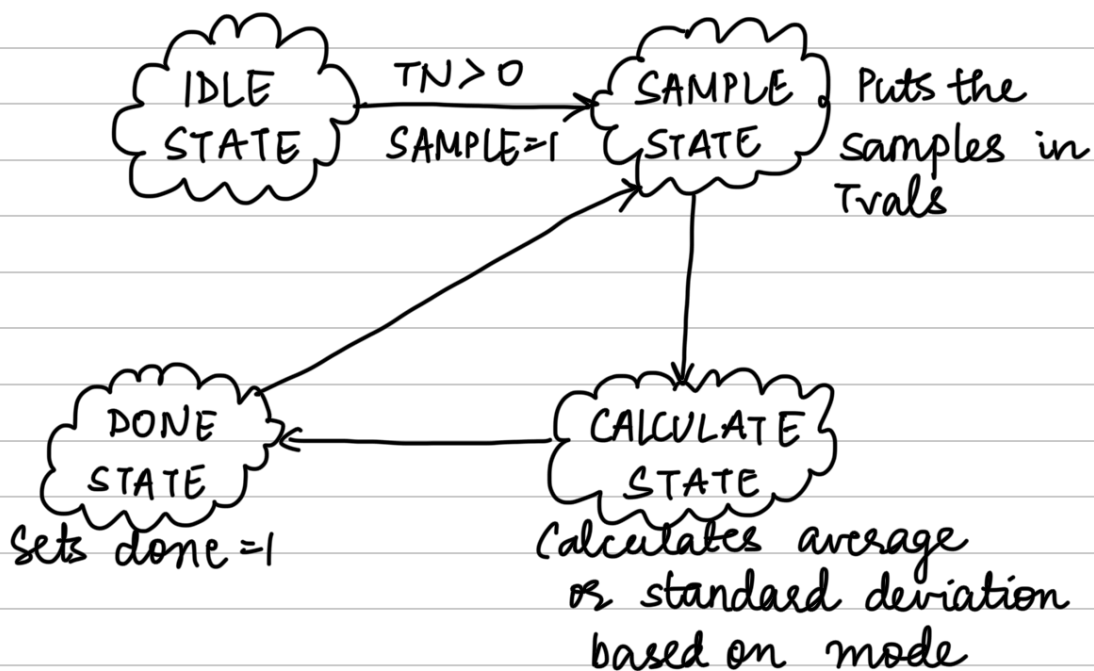
Total Power

```

-----
Total Internal Power:      0.62333379      38.4594%
Total Switching Power:    0.82712880      51.0334%
Total Leakage Power:      0.17029660      10.5072%
Total Power:              1.62075919
-----

```

Schematic:



Timing Report:

```

-----
(clock CLK)          capture          20000 R
-----
Cost Group   : 'CLK' (path_group 'CLK')
Timing slack :      0ps
Start-point  : count_reg[6]/CP
End-point    : sd_reg[11]/SE

```

Area Report:

```
legacy_genus:/> report area
```

```
=====
Generated by:      Genus(TM) Synthesis Solution 22.16-s078_1
Generated on:      Dec 13 2024 10:34:21 am
Module:           final_project_fsmm
Technology library: tcbn65gpluswc 121
Operating conditions: WCCOM (balanced_tree)
Wireload mode:    segmented
Area mode:        timing library
=====
```

```
-----
Instance      Module  Cell Count  Cell Area  Net Area  Total Area  Wireload
-----
final_project_fsmm      6544  16859.520      0.000    16859.520 ZeroWireload (S)
(S) = wireload was automatically selected
```

Power Report:

```
Instance: /final_project_fsmm
Power Unit: W
PDB Frames: /stim#0/frame#0
```

```
-----
Category      Leakage      Internal      Switching      Total      Row%
-----
memory        0.000000e+00  0.000000e+00  0.000000e+00  0.000000e+00  0.00%
register      1.56707e-05   4.57837e-05   5.74656e-06   6.72010e-05   10.67%
latch        0.000000e+00  0.000000e+00  0.000000e+00  0.000000e+00  0.00%
logic        1.25596e-04   2.31952e-04   2.00424e-04   5.57971e-04   88.62%
bbox         0.000000e+00  0.000000e+00  0.000000e+00  0.000000e+00  0.00%
clock        2.04447e-07   1.45971e-06   2.81177e-06   4.47592e-06   0.71%
pad          0.000000e+00  0.000000e+00  0.000000e+00  0.000000e+00  0.00%
pm           0.000000e+00  0.000000e+00  0.000000e+00  0.000000e+00  0.00%
-----
Subtotal      1.41471e-04   2.79195e-04   2.08982e-04   6.29648e-04  100.00%
Percentage    22.47%       44.34%       33.19%       100.00%  100.00%
-----
```

Clock Period (ns)	20
Slack (ns)	0ps
Average Power Consumption (mW)	0.629
Area (micrometer^2)	16859.530
average energy consumption per temperature reading(mW)	0.02097
latency (i.e., the # of clock cycles it takes to generate AVG/SD after SAMPLE is asserted)	2 clock cycles

Throughput (i.e., the maximum # of temperature readings the circuit can handle in 1 second)	40
---	----

Conclusion, Discussions, and Comparison:

The project aimed to design a Verilog system to compute the moving average and moving standard deviation of temperature samples provided by the USDA. The design featured inputs for mode selection 0 for average and 1 for standard deviation, reset, temperature, and clock, as well as outputs to indicate when the system was ready for a new sample, when computations were complete, and the computed value average or standard deviation. The initial implementation used a simple shift register to manage a window of temperature samples. This approach computed the sum for the average and variance for the standard deviation, with adjustments made to the calculations to mitigate rounding errors and ensure accurate outputs. The primary goal of the initial implementation was to create a straightforward, functional design that could serve as a baseline for optimization.

A finite-state machine was implemented to enhance the design to replace the shift register. The FSM introduced a structured approach that reduced both area and power consumption by minimizing unnecessary switching activity, leveraging low-power idle states, enabling clock gating to deactivate unused components, and reusing logic to simplify the design. These changes significantly reduced the area, as the FSM required fewer dedicated storage elements and logic gates compared to the shift-register-based implementation. Additionally, the FSM reduced power consumption by controlling transitions and using idle states efficiently. Despite these benefits, the FSM implementation was more complex to design and verify, requiring additional development time compared to the straightforward initial approach.

In conclusion, while the initial design provided a functional baseline, the FSM-based implementation brought significant improvements in area and power efficiency.

Feedback:

While this project was manageable, it was frustrating to actually get it to work. We took one lecture to learn how to program in verilog and that was it. We never learned troubleshooting skills, and most teams were having trouble just getting there regular code to work. For this entire semester, we only coded two labs, so we never got practice. For future classes, I would put more emphasis on coding in labs.